

Management and Content Delivery for Smart Networks: Algorithms and Modeling

Academic Year 2025-2026

L2: Aerial Edge Computing for remote environmental monitoring

1 Background

Environmental monitoring in remote natural areas is becoming increasingly important for biodiversity conservation, anomaly detection and in general the protection of ecosystems that are difficult to observe continuously through human intervention alone. In these context, distributed sensing nodes can be deployed to collect environmental data such as acoustic recordings, images, temperature, humidity or other ecological signals. The collected information can then be analyzed to detect events of interest, classify relevant patterns, trigger adaptive actions or protective measures in the monitoring system.

A major challenge in this scenario is that sensing nodes are typically energy-constrained and equipped with limited computational capabilities. As a consequence, processing tasks entirely at the device may lead to long execution times for computationally intensive operations, queue buildup when the task arrival rate increases, and excessive energy consumption at the sensing node. These limitations are particularly problematic for those services that require a timely reaction to critical or unusual events. At the same time, remote natural areas are often not covered by reliable terrestrial telecommunication infrastructure. This makes it difficult to rely on traditional cloud-based solutions for data transmission and processing.

In this context, *Non-Terrestrial Networks (NTNs)*, which are expected to play a key role in future *6G systems*, represent a promising solution to provide both connectivity and computing capabilities in infrastructure-less environments. Among NTN technologies, *High Altitude Platform Stations (HAPS)* offer a particularly attractive trade-off between coverage, latency and continuous availability. HAPSs are typically represented by floating airships or aircrafts operating at altitudes of approximately 20 km, thus providing wide-area coverage.

To address the challenges associated with environmental monitoring in remote areas, we consider the integration of a HAPS equipped with edge computing capabilities. The HAPS can act as an aerial processing node, continuously available to support remote sensing nodes by executing offloaded tasks. Compared to local processing, HAPS-edge computing can reduce the time needed to process complex tasks and can relieve the sensing node from part of the computational burden. However, offloading is not always beneficial, since it also introduces communication delay, transmission energy consumption for the distributed devices and possible congestion when many sensing nodes simultaneously forward tasks to the HAPS.

The goal of this laboratory is to investigate this trade-off and understand under which conditions local processing, HAPS-edge computing or hybrid strategies become preferable.

2 Scenario

We consider a remote environmental monitoring system composed of a set of sensing nodes and a HAPS equipped with edge computing capabilities. The considered HAPS-edge computing scenario is illustrated in Fig. 1. A set of M sensing nodes (microphones, cameras, temperature

and humidity sensors...) collect environmental data and generate computational tasks, that can either be processed locally or offloaded to the HAPS. The HAPS is modeled as an edge computing platform (aerial mini-server farm) composed of multiple processing units (N servers) sharing a common buffer. Incoming tasks are stored in a shared buffer and assigned to the first available server in a First In First Out (FIFO) fashion, enabling parallel processing.

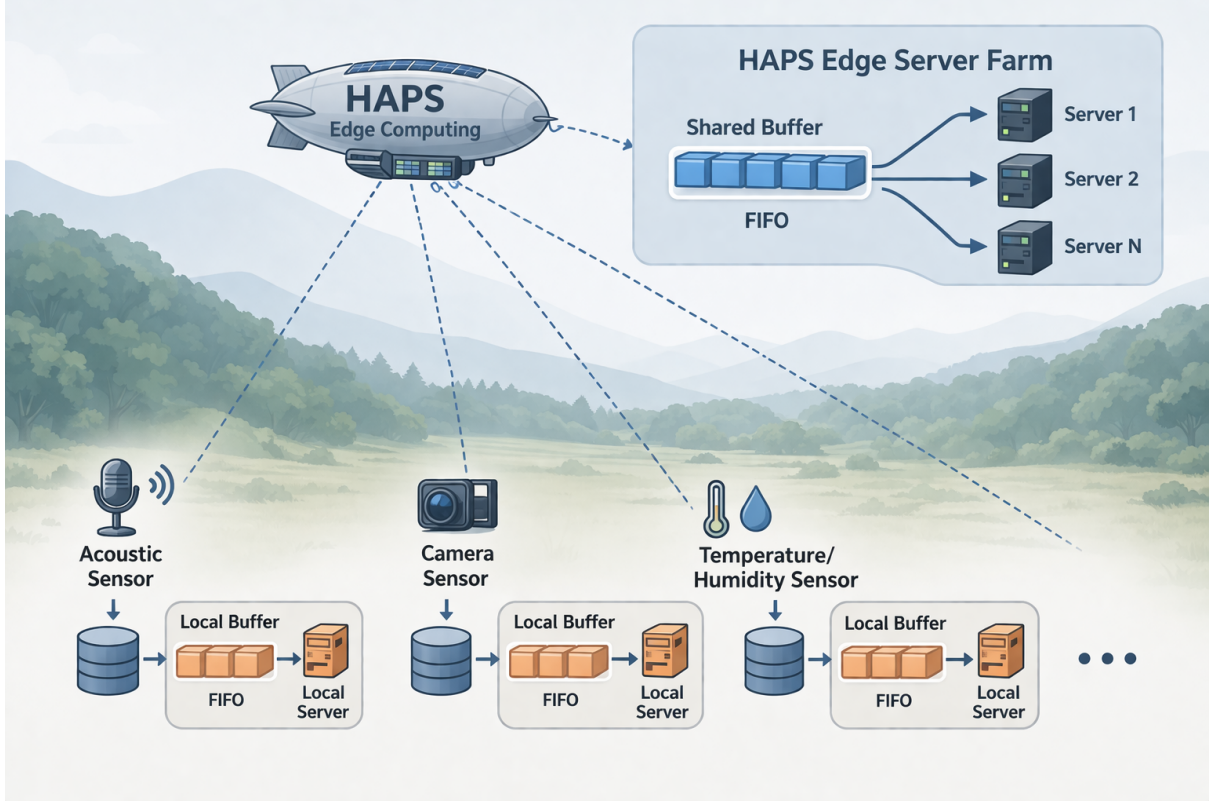


Figure 1: HAPS-edge computing scenario for environmental monitoring.

The generated tasks may differ not only in their *data size*, but also in their *computational demand*. For example, some tasks may involve lightweight threshold-based processing, while others may require more demanding machine learning inference. Furthermore, different types of tasks correspond to different services and may therefore exhibit heterogeneous requirements. In particular, the following distinction should be considered:

- **Delay-sensitive tasks**, which require timely processing. These tasks may correspond, for instance, to early fire or smoke event detection, gunshot or chainsaw detection in protected areas, poaching-risk alerts, recognition of distress calls from endangered species or immediate reconfiguration of the sensing node when a suspicious environmental pattern is detected.
- **Delay-tolerant tasks**, which are associated with non-urgent background activities, such as long-term biodiversity monitoring, periodic environmental data analysis, archival processing or the bulk analysis of recorded content.

Each generated task can be executed entirely at the sensing node (**Local processing**) or it can be offloaded to the HAPS and processed there (**HAPS-edge processing**). The task heterogeneity creates a non-trivial trade-off between local processing and task offloading.

3 Laboratory activity

The goal of this laboratory is to investigate how computational tasks generated by sensing nodes should be processed in a HAPS-edge computing system. More specifically, you are asked to study how different processing strategies affect delay, energy consumption, queueing behavior and Quality of Service (QoS) satisfaction.

The analysis is organized into two parts. In the first part (Part A), you will focus on system behavior and offloading strategies. In the second part (Part B), you will study more advanced aspects, including coexisting heterogeneous task classes, adaptive offloading strategies, chunk based processing, QoS-aware scheduling and offloading policies. Throughout the analysis, you are expected to support your claims with plots derived from simulation results and to explain the observed trends in light of the adopted model assumptions.

Guidelines to model the system

To support the simulation analysis, you may adopt a simplified system model that captures the main elements of the problem without introducing unnecessary complexity.

(a) Task characterization. Tasks are generated over time according to an arrival process with average rate λ [tasks/s]. Each task is characterized by three main parameters:

- input data size L [bit], representing the amount of data to be transmitted and/or processed;
- computational demand C [CPU cycles], representing the processing effort required by the task;
- latency constraint $D^{\max}$ [s], representing the maximum acceptable completion time.

To better reflect realistic scenarios, tasks should not all be identical. Instead, at least two task classes should be considered, for instance:

- **delay-sensitive tasks** (K_s), with average size L_s and computational demand C_s , which require low latency and typically correspond to real-time monitoring services; the latency constraint is represented by $D^{\max}$
- **delay-tolerant tasks** (K_t), with average size L_t and computational demand C_s , which can sustain longer delays and correspond to background or non-urgent processing.

Each task can be either processed locally at the sensing node or offloaded to the HAPS. When a task is processed locally, its completion time depends on the processing capability of the device (denoted as f_{loc}) and on the current load at the node. In particular, when multiple tasks are present, queueing effects should be taken into account. When a task is offloaded to the HAPS, its completion time is determined by multiple components: the time required to transmit the task over the uplink (depending on the data rate R_{ul} and the task size L), the queueing delay at the HAPS (denoted as $W^{(haps)}$), the processing time at the HAPS (depending on its computational capability f_{haps}) and the propagation delay t_p .

For simplicity, the downlink transmission of results can be neglected, assuming that the size of the response is negligible compared to the input data.

(b) Energy considerations. If energy consumption is considered, a simple model is sufficient. In particular, local processing energy can be assumed proportional to the time spent executing the task, while transmission energy can be assumed proportional to the

amount of data transmitted. This allows capturing the main trade-off between local computation and offloading without introducing unnecessary complexity.

(c) QoS considerations. Each task is associated with a latency constraint $D^{\max}$, which defines the maximum acceptable completion time. A task can be considered successful if it is completed within this limit.

To evaluate system performance, you should consider metrics that capture both average behavior and QoS satisfaction, such as average delay, delay measured separately for each task class, deadline violation probability, system load indicators (buffer occupancy, utilization)...

Part A - System Behavior and HAPS Offloading

Task 1:

Consider a baseline scenario in which a set of M sensing nodes generates computational tasks that are processed locally at each device. Each sensing node can be modeled as a system with one server and a local buffer. Assume that tasks arrive according to a stochastic process with constant average rate λ .

- 1.a - Focusing on a single sensing node, observe the system behavior during the warm-up transient period and identify the transition to steady-state conditions. Explain how the transient phase can be recognized from the evolution of relevant performance metrics (e.g., delay, queue length).
- 1.b - Apply a suitable method to remove the warm-up transient from your simulations and justify your choice.
- 1.c - Once steady-state conditions are reached, analyze the system performance under different workload levels (i.e., different values of λ).

Task 2:

Now, consider the integration of a HAPS equipped with edge computing capabilities, acting as a mini edge server farm with multiple processing units and a shared buffer.

Each sensing node can offload a fraction of its tasks to the HAPS. Offloading decisions can be modeled in a simplified way by introducing a parameter $p \in [0, 1]$, representing the probability that an incoming task is offloaded to the HAPS, while the remaining fraction $(1-p)$ is processed locally.

You can therefore progressively move from:

- $p = 0$: fully local processing (baseline scenario);
- $0 < p < 1$: partial offloading;
- $p = 1$: full offloading to the HAPS.

Let us assume that all the sensing nodes are of the same type, thus generating exclusively delay-tolerant tasks (K_t) or delay-sensitive tasks (K_s).

- 2.a - Starting from the baseline scenario ($p = 0$), gradually increase the offloading probability p . Analyze how system performance evolves as more tasks are offloaded to the HAPS, comparing the cases in which either delay-tolerant tasks only or delay-sensitive tasks only are generated.
- 2.b - Identify operating regions in which offloading is beneficial and regions in which it degrades performance. Explain the observed trends by considering the trade-off between local processing limitations (including energy budget constraints) and HAPS congestion.
- 2.c - Compare the extreme cases $p = 0$ and $p = 1$. Discuss under which conditions full offloading is advantageous or detrimental, taking into account both computation and communication aspects.

Tip

When discussing performance results, it is important to distinguish among transmission delay, propagation delay, queueing delay and processing delay. These components may evolve differently depending on the chosen processing strategy.

Part B - Advanced Offloading and QoS-Aware System Design

In this part, you will move from system understanding to system design, considering more advanced scenarios in which heterogeneous task classes coexist and Quality of Service (QoS) requirements must be explicitly taken into account.

Task 3:

So far, sensing nodes have been assumed to generate homogeneous tasks (either delay-sensitive or delay-tolerant). In this task, we consider a more realistic scenario in which **both task classes coexist**, with different proportions.

Let us denote by $\alpha \in [0, 1]$ the fraction of delay-sensitive tasks, while $(1 - \alpha)$ represents the fraction of delay-tolerant tasks.

- 3.a - Consider different values of α (e.g., low, medium and high proportion of delay-sensitive tasks) and analyze how system performance is affected under partial offloading (i.e., for different values of p). In particular, evaluate:
 - delay experienced by each task class;
 - fraction of tasks that satisfy their latency constraint;
 - overall system load (e.g., buffer occupancy, utilization).

Discuss how the coexistence of heterogeneous tasks affects the trade-off between local processing and HAPS offloading.

- 3.b - **Optional (up to 1 point):** Design a simple **QoS-aware offloading strategy** that aims at improving the performance of delay-sensitive tasks. For example, the strategy may prioritize delay-sensitive tasks for local processing or offloading, or dynamically adjust the offloading probability p depending on the task type or system load, or apply different policies for different task classes.
 - Evaluate the effectiveness of your strategy in terms of delay and QoS satisfaction for each task class, impact on delay-tolerant tasks and overall system performance.

- Additionally, investigate how system design parameters affect performance. In particular, analyze the impact of:
 - the number of HAPS servers (N);
 - the processing capability of the HAPS (f_{haps});
 - the size of the shared buffer at the HAPS.

Discuss how these parameters influence system behavior and the effectiveness of your QoS-aware strategy.